

From assistant to accomplice: A forensic framework for CLI coding agents in incident response

Taewoo Kim, Doowon Jeong*

Department of Forensic Sciences, Sungkyunkwan University, Seoul, South Korea

ARTICLE INFO

Keywords:

CLI Coding Agents
Digital Forensics
Forensic Framework
Artefact Analysis
Incident Reconstruction

ABSTRACT

CLI coding agents are increasingly being adopted as agentic systems that integrate code generation, filesystem access, command execution, and external service interaction. This integration introduces new challenges for digital forensic analysis, particularly given the complex interplay between these agents and their surrounding execution environments. In this paper, we present a forensic framework for analysing CLI coding agent environments by interpreting artefacts generated by these agents through a multi-layer evidence perspective. We identify and categorise key artefacts critical to incident response, and conduct a comparative analysis of representative CLI coding agents with a focus on their artefact generation characteristics. Building on this analysis, we propose a structured investigation process that supports systematic artefact collection and interpretation. We further validate the proposed framework through case studies demonstrating how incident scenarios can be reconstructed from the identified artefacts. Our findings reveal that while certain artefacts enable partial reconstruction of agent-driven activities, comprehensive incident analysis requires correlating evidence across multiple sources, highlighting both the promise and the inherent limitations of forensic analysis in CLI coding agent environments.

1. Introduction

Recently, Large Language Model (LLM)-based CLI coding agents have been widely adopted in software development. Operating within command-line environments, these agents provide an integrated set of capabilities, including filesystem access, command execution, and interaction with external services, that collectively streamline development workflows [1]. As a result, development productivity has improved considerably and coding tasks have become more accessible even to users without specialised expertise in development or security.

However, these capabilities come with new security challenges. As the code generation capabilities of such agents advance, adversaries can exploit techniques such as jailbreaking to induce agents to generate or distribute malicious scripts [2]. Beyond direct manipulation, attacks such as prompt injection or directory listing can cause agents to execute commands that deviate from the user's original intent, potentially leading to unintended security-relevant actions [3]. Unlike attacks targeting cloud-based models, these agent-assisted attacks directly affect the local execution environment of the user's system, a characteristic that carries significant forensic implications.

Modern CLI coding agents further complicate this picture by supporting extensibility features such as MCP servers, Hooks, and Skills [4, 5, 6]. Although these mechanisms enhance the utility of agents, they also introduce pathways through which external resources and execution environments can become embedded within the agentic system. This structural openness expands the attack surface and enables new classes of supply-chain attacks [7]. Despite these developments, the digital forensics community has

yet to systematically address the investigation of LLM-based agentic systems following security incidents. Open questions include how relevant artefacts are identified, how agent-driven behaviour is reconstructed, and how interactions involving external resources and execution processes are interpreted.

In this paper, we present a forensic framework for analysing CLI coding agent environments and validate its applicability through case studies. Because CLI coding agents operate by integrating user interaction, external resources, and command execution within a single workflow, they pose distinctive challenges for digital forensic analysis. To address these challenges, we examine how artefacts are generated and distributed across the different virtual layers of agent operation, and identify the types of evidence that are recoverable for forensic purposes. Building on this analysis, we develop a structured framework that supports the systematic identification, collection, and interpretation of artefacts in CLI coding agent environments.

We further validate the proposed framework through representative incident scenarios, demonstrating how attacks involving external resources and execution processes can be reconstructed from available artefacts. Our findings reveal both the potential and the limitations of forensic analysis in such environments, particularly with respect to visibility and reconstruction completeness.

The main contributions of this paper are as follows:

- We propose a forensic framework for CLI coding agent environments, focusing on artefact identification and analysis across different operational layers.
- We analyse the artefact generation characteristics of representative CLI coding agents and categorise them from a forensic perspective.

*Corresponding author

 twotae2000@skku.edu (T. Kim); doowon@skku.edu (D. Jeong)

- We present case studies of incident scenarios and demonstrate how such incidents can be reconstructed using the proposed framework.

The remainder of this paper is organised as follows. Section 3 presents the architecture of CLI coding agents and characterises their operational behaviour. Building on this foundation, Section 4 introduces a forensic framework tailored to CLI coding agent environments, comprising a multi-layer evidence model and a structured investigation process for systematic artefact analysis. Section 5 provides an empirical analysis of forensic artefacts generated by representative CLI coding agents, characterising their evidential properties across different layers. Section 6 presents case studies of incident scenarios and demonstrates how such events can be reconstructed using the proposed framework. Finally, Section 7 concludes the paper by summarising key findings, discussing limitations, and outlining directions for future research.

2. Related work

With the increasing adoption of LLM-based agentic systems, research on their security and reliability has gained considerable attention. The autonomous execution capabilities of these agents, combined with their reliance on external resources, have given rise to new attack vectors, prompting a growing body of work that examines these risks from both adversarial and defensive perspectives. In parallel, forensic research has begun to explore how data generated by LLM-based applications can serve as a source of digital evidence.

2.1. Security in LLM-based Agentic Systems

Prior work has shown that as Large Language Models evolve into autonomous agents capable of interacting with external environments, they become exposed to a substantially broader range of security threats than their standalone counterparts [8]. Saha et al. proposed JAWS-BENCH and demonstrated that coding agents present a markedly higher risk of generating malicious code through jailbreaking compared to standalone models [9]. Kumar et al., through the BrowserART benchmark, showed that even models trained to refuse harmful instructions in chatbot settings may exhibit significantly degraded safety behaviour when deployed as browser agents [10]. Badhe introduced the ScamAgent framework, showing that an agent’s multi-turn planning and goal decomposition capabilities can be exploited to circumvent existing safety filters and automate real-world scam scenarios [11]. Chiang et al. took a structural view of this growing vulnerability, attributing it to factors inherent in agentic design, including the embedding of user goals into system prompts, step-by-step execution, and dynamic observation capabilities [12].

Efforts to mitigate these agent-specific threats have produced a range of security mechanisms operating at both the architectural and runtime levels. Wang et al., for example, introduced a domain-specific language framework that enforces user-defined safety policies during agent execution,

demonstrating its effectiveness in blocking potentially harmful actions such as unsafe code execution [13].

Evaluation frameworks for systematically assessing such mechanisms have also emerged; DeBenedetti et al. proposed AgentDojo, which tests agent susceptibility to prompt injection across realistic task scenarios and untrusted data environments [14].

Despite this progress, existing studies have largely concentrated on red-teaming exercises or improving agent robustness, leaving the question of agentic systems as attack surfaces themselves comparatively underexplored. Lee et al. represent a notable exception, presenting an end-to-end attack in Visual Studio Code extensions and agent-integrated IDEs that encompasses both command execution and data exfiltration [3]. Their work establishes the viability of coding agents as attack surfaces; however, the analysis remains limited to a specific development environment. Broader systems such as CLI coding agents, which operate with wider execution privileges and a more expansive attack surface, have yet to receive comparable scrutiny. Unlike conversational or browser-based agents, CLI coding agents have direct access to the filesystem and execution environment, giving rise to a richer and more structurally diverse set of forensic artefacts.

2.2. Digital Forensics for LLM-based Applications

Digital forensic research on LLM-based applications has emerged as an active area relatively recently. Cho et al. conducted an artefact analysis of conversational AI systems, including ChatGPT, Gemini, Copilot, and Claude, and proposed a forensic investigation framework grounded in those findings [15]. Similarly, Tyagi et al. examined forensic artefacts produced by ChatGPT, Copilot, and Gemini in mobile environments [16]. Taking a more structural approach, Jeong et al. proposed a forensic framework that categorises LLM applications into three components (backend runtime, client interface, and integrated platform) and organises artefact analysis accordingly [17]. More recent work has extended this line of inquiry to AI companion applications, examining their forensic artefacts alongside associated privacy vulnerabilities [18].

Collectively, however, these studies have approached LLM applications primarily from a user-centric forensic perspective, concentrating on user activity traces while giving limited attention to agentic systems as potential attack surfaces in their own right. Forensic analyses that jointly account for both the occurrence and reconstruction of attacks in *open-world agentic systems*, defined here as environments where external resources and execution capabilities are tightly integrated, remain largely absent from the literature. These gaps underscore the need for a structured forensic framework capable of systematically organising and interpreting artefacts across multiple layers to support reliable incident reconstruction.

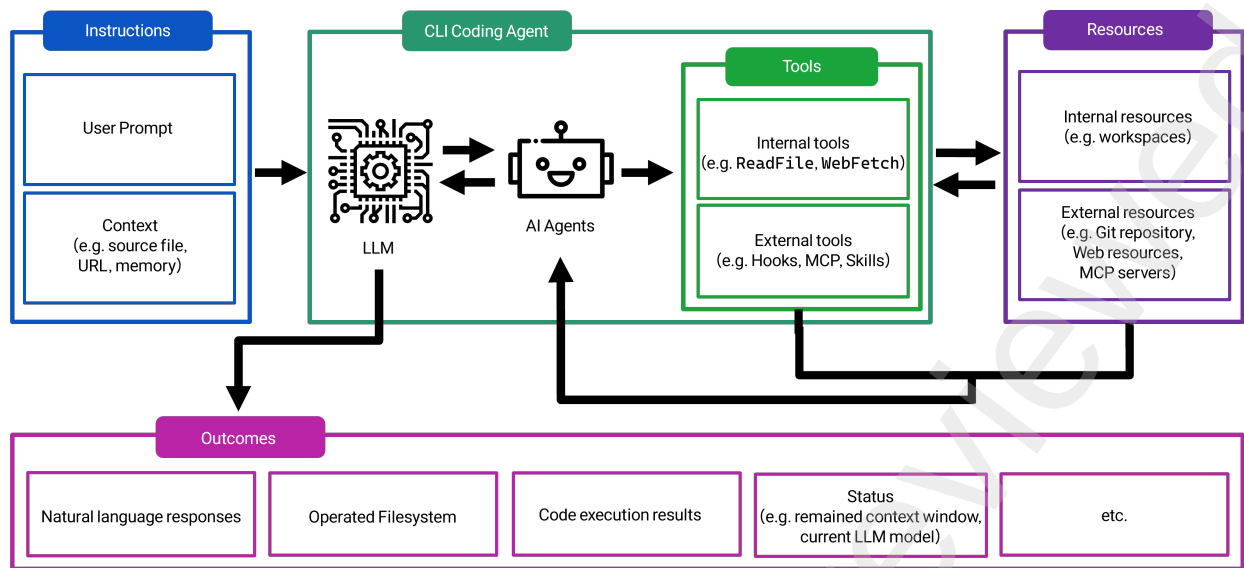


Figure 1: Abstract architecture of CLI coding agents used in this study.

3. System model of CLI coding agents

To enable systematic forensic analysis of incidents involving CLI coding agents, we first define an abstract architecture that captures the core components and their interactions within the agent environment (see Figure 1). Although modern CLI coding agents are frequently updated, with features added or revised across releases, the components described here remain consistent across such changes.

3.1. Architecture of CLI coding agent

Although individual CLI coding agents differ in their specific features and implementation details, their core operational characteristics are widely shared. We therefore organise the agent architecture around five common components.

Instructions. The instructions refer to the input provided to the LLM and represent the starting point of user-agent interaction. Users typically supply natural-language prompts, though the LLM may also incorporate additional context when interpreting these instructions, such as the contents of source files, referenced URLs, or information retained from prior interactions.

LLM & AI agents. The LLM serves as the core of the agent’s decision-making process. Given a set of user instructions, it interprets the request, determines an appropriate course of action, and coordinates the execution of tasks through AI agents. Once a workflow is completed, the LLM produces a response that reflects both the execution results and the original user query.

Tools. The tools constitute the set of utilities available to the agent to perform concrete actions. By default, agents provide built-in tools such as ReadFile and WebFetch, which support operations including filesystem access and command

execution. Users may extend this set by registering custom tools, for example, by adding MCP servers or defining Hooks that invoke user-defined scripts. These tools serve as the primary interface through which the agent translates high-level instructions into executable actions.

Resources. CLI coding agents can access a range of resources during execution. The agent operates within a workspace typically defined by the current project directory and can interact with files and directories within that scope. In the absence of sandboxing, this access may extend beyond the project boundary to the broader host system. Agents can also interact with external resources, including web content, Git repositories, APIs, and MCP servers, through available tools. Such resources shape agent behaviour by supplying information or functionality that may be incorporated into the execution process.

Outcomes. Agent actions produce outcomes in multiple forms: responses presented to the user through the command-line interface, as well as changes to the system state such as file modifications or command execution results. Together, these outcomes constitute the observable effects of agent behaviour within the execution environment.

3.2. Execution workflow

CLI coding agents are typically launched from a terminal within a project directory, which serves as the primary execution context. Users interact with the agent by issuing instructions that are then interpreted and performed through a sequence of internal operations and tool interactions.

Execution behaviour may be shaped by runtime configuration options, for example, those governing sandboxing or per-action approval prompts. Gemini CLI, for instance, provides a `--sandbox` flag for container-based isolation via

Docker or Podman, and a `--yolo` flag to bypass user confirmation for command execution [19]. Sandboxing may restrict the scope of accessible resources, while approval bypass mechanisms allow actions to proceed without explicit user interaction.

3.3. Evidence scope

This study considers a forensic investigation scenario in which potentially malicious behaviour arises through interactions with external resources, such as Git repositories containing embedded harmful payloads or malicious MCP servers distributed through unofficial channels.

From a digital forensics standpoint, we assume that the investigator has access to artefacts generated on the local system, encompassing both agent-level data (e.g. session history, configuration files, and tool invocation records) and system-level data (e.g. process execution logs and filesystem traces). Direct access to attacker-controlled external infrastructure, such as remote servers or external databases, is not assumed.

This assumption reflects realistic investigative conditions, in which evidence must be reconstructed primarily from locally available artefacts. Consequently, the visibility of attack behaviour depends on how interactions with external resources are recorded within the agent and the host system, which in turn directly affects the traceability of such incidents.

This study adopts a post-incident forensic perspective, assuming that the investigation is conducted after an attack has occurred and without prior instrumentation. Under such conditions, volatile data sources, including network traffic captures and memory dumps, are typically unavailable. The focus therefore falls on persistent artefacts recoverable from the local system, spanning both agent-level and system-level data, in keeping with the practical constraints faced by digital forensic investigators.

4. Forensic investigation framework

For analytical clarity, we conceptualise CLI coding agents through a multi-layered abstraction comprising four distinct layers: the interaction layer, operation layer, execution layer, and artefact layer. These layers are not inherent components of the agent architecture; rather, they are analytically defined constructs introduced to support systematic forensic reasoning. This abstraction is designed to capture how agent behaviour propagates across layers and how observable evidence is produced at each stage. Crucially, agent behaviour is not confined to any single layer; it can span multiple layers simultaneously, and this cross-layer propagation directly determines the availability, location, and structure of the resulting forensic artefacts.

4.1. Multi-layer evidence model

We organise artefacts generated by CLI coding agents into a layered structure, in which different artefact types are associated with different operational levels (see Figure 2).

A central location for storing agent-level data is the *configuration directory*, typically residing under a hidden directory in the user's home path. This directory holds a variety of artefacts produced during agent operation, including session logs, configuration files, tool-related records, and user-specific data such as authentication information, and thus serves as the primary source of agent-level evidence in forensic analysis.

4.1.1. Layer segmentation

Interaction layer. This layer captures artefacts related to the interaction between the user and the agent, including user instructions, contextual information, and model responses. These interactions are typically recorded as session logs or history files within the configuration directory, providing primary evidence of user intent and high-level agent behaviour.

Operation layer. This layer encompasses artefacts generated during the agent's internal operations: tool invocation records, command traces, and action logs produced as the agent processes user instructions. Because these artefacts are typically co-recorded with interaction layer data in session logs, they are generally located within the same configuration directory.

Resource layer. This layer captures artefacts associated with resources accessed by the agent, including retrieved external content (e.g. web data and repository files) and configuration data defining connections to external systems such as MCP servers or plugins. Such artefacts may appear in configuration files (e.g. `settings.json` or `settings.toml`), project directories, or session records, and can reflect both benign and potentially malicious inputs.

Host system layer. This layer encompasses artefacts generated at the operating system level as a direct consequence of agent execution. These include process logs, system logs (e.g. audit logs), and filesystem artefacts reflecting file creation, modification, or deletion, which together capture the observable effects of agent actions on the host system.

4.1.2. Evidence class

Artefacts observed across these layers are further categorised into two evidence classes: *agent-level* and *system-level*. Agent-level artefacts originate from within the CLI coding agent environment, encompassing session logs, configuration files, and tool invocation records stored in the agent's configuration directory. System-level artefacts originate from the host operating system, comprising process execution logs and filesystem traces. This distinction characterises differences in forensic visibility across layers.

Of particular forensic significance are discrepancies between the two classes: divergences between agent-level and system-level artefacts may indicate execution paths that are not fully observable within the agent environment alone.

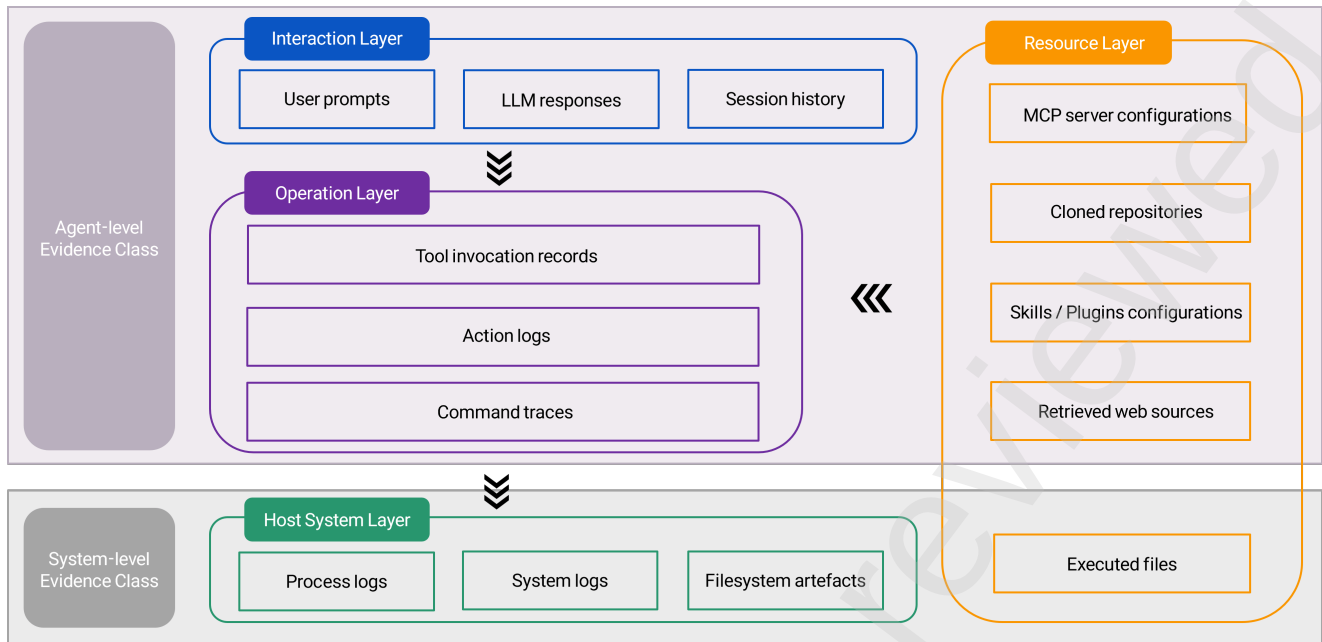


Figure 2: Multi-layer evidence model.

Table 1

Mapping between architectural components, layers, artefacts, and evidence classes.

Component (Section 3.1)	Layer (Section 4.1)	Evidence Class	Typical Artefacts
Instructions	Interaction	Agents	Session history, user prompt
LLM responses	Interaction	Agent	Generated outputs, dialogue logs
Tools	Operation	Agent	Tool invocation records, command traces
Resources (internal)	Resource	Agent / System	Project files, directory contents
Resources (external)	Resource / Operation	Agent / System	Configured MCP server, Hooks, Plugins, retrieved data
Command execution	Host system	System	Process execution logs, audit logs
Other Outcomes	Interaction / Host system	System	Filesystem changes, execution results

4.2. Artefact mapping

To support systematic forensic analysis, we map the architectural components defined in Section 3.1 to the multi-layer evidence model introduced in Section 4.1. This mapping clarifies how each component contributes to the generation of artefacts across layers (see Table 1).

This mapping reveals that artefacts are not generated uniformly across the system; instead, their distribution across layers depends on how each component participates in the execution workflow. Effective forensic analysis therefore requires correlating artefacts across multiple layers, rather than relying on any single source of evidence.

Note that artefact generation does not necessarily follow a complete or uniform path across all layers. In practice, different types of agent behaviour may leave traces at only a subset of layers.

For example, certain interactions remain confined to the interaction layer. When the agent processes a user instruction and returns a response without invoking external tools or executing system-level commands, the resulting artefacts are limited to session history and dialogue records. Similarly,

even when the agent invokes an external resource such as an MCP server, if that server executes only script-language code, the interaction leaves no corresponding traces at the host system layer.

In contrast, some execution paths may bypass agent-level visibility. For example, commands triggered through pre-configured Hooks or external scripts may execute directly at the host system layer without leaving explicit traces in interaction or operation layer artefacts. In such cases, the behaviour is observable only through system-level evidence such as process logs or filesystem artefacts.

These discrepancies illustrate that artefact visibility is inherently non-uniform across layers. Reconstructing an incident consequently requires correlating evidence from both agent-level and system-level sources while accounting for potential gaps where certain execution paths are not captured within specific layers.

4.3. Investigation process

Building on the multi-layer evidence model, we define a structured investigation process for analysing security incidents in CLI coding agent environments. The process reflects established digital forensic practice while accounting for the multi-layered nature of agent execution.

1. *Identification of contexts.* The investigator first establishes the scope of the incident, identifying the relevant project environment, agent configuration, and any external resources, such as Git repositories or MCP servers, referenced in configuration files (e.g. `settings.json` or `settings.toml`). Beyond conventional artefacts such as authentication or kernel logs, the presence of CLI coding agents should be explicitly considered, as their configuration and integration with external resources may introduce additional attack surfaces. This step establishes the contextual boundaries within which subsequent artefacts are interpreted.
2. *Artefact collection.* Artefacts are collected from both agent- and system-level sources. Agent-level artefacts, including session histories, configuration files, and tool invocation records, are typically obtained from the agent's configuration directory. System-level artefacts, including process execution logs, filesystem traces, and audit logs, are collected from system log directories. The collection focuses on persistent artefacts available after the incident has occurred.
3. *Multi-layer correlation.* The collected artefacts are analysed across layers to identify relationships among user inputs, resource interactions, and executed commands. This step involves correlating interaction records (e.g. prompts and LLM reasoning traces), execution traces (e.g. tool invocation logs), and system events (e.g. command execution entries).
4. *Reconstruction of events.* Building on the relationships identified in the preceding step, the investigator assembles a coherent sequence of events by integrating artefacts across layers. The goal is to infer causal chains linking external inputs, agent actions, and resulting system-level effects, ultimately producing an interpretable account of the incident.
5. *Consistency verification.* The reconstructed sequence is systematically validated against the underlying artefacts to confirm internal consistency and evidential support. This involves checking temporal coherence, verifying alignment between command traces and recorded actions, and confirming the consistency of resource references across layers. Detected inconsistencies may signal missing artefacts, evidence of tampering, or execution paths not captured within the available evidence, each of which warrants further investigation or the consideration of alternative hypotheses.

This process underscores the importance of cross-layer analysis: no single artefact source is sufficient to fully account for agent behaviour in complex execution scenarios.

4.4. Incident reconstruction

The outcome of the investigation depends on the availability and completeness of artefacts across layers. We define three levels of reconstruction based on the degree to which the incident can be inferred from the evidence collected.

- *Fully reconstructable.* All key stages of the incident, including user input, resource interaction, command execution, and resulting system changes, can be identified and linked through available artefacts, and the causal chain of the incident is clearly established.
- *Partially reconstructable.* Some stages of the incident can be identified, but gaps remain in the causal chain due to missing or incomplete artefacts. Command execution, for example, may be observable without a clear link to the originating input or resource, particularly in the absence of logs in the configuration directory, as discussed in further detail in Section 5.
- *Non-reconstructable.* The critical stages of the incident are not observable from the available artefacts, preventing reliable reconstruction. This typically occurs when execution events are not recorded in agent-level artefacts and cannot be recovered from system-level logs.

These categories provide a basis for evaluating forensic traceability in CLI coding agent environments and are applied in subsequent sections to compare the reconstructability of different attack scenarios.

5. Comparative analysis of forensic artefacts

This section applies the forensic investigation framework introduced in Sections 3 and 4 to analyse artefacts generated by representative CLI coding agents. The analysis focuses on how agent-level artefacts are distributed across layers and how such differences affect forensic visibility, traceability, and the capacity to reconstruct agent behaviour. Throughout, findings are interpreted in relation to the multi-layer evidence model and evidence classes defined in Section 4, enabling a systematic assessment of how artefacts from different layers contribute to cross-layer incident reconstruction.

5.1. Experimental setup

In this study, three representative CLI coding agents were selected for analysis: Anthropic's Claude Code [20], OpenAI's Codex CLI [21], and Google DeepMind's Gemini CLI [22]. All three operate in command-line environments and, despite differences in implementation, exhibit broadly similar architectural characteristics and artefact generation mechanisms.

Table 2

Locations of agent-level artefacts across layers.

Artefact Type	Claude Code (v2.1.72)	Codex CLI (v0.113.0)	Gemini CLI (v0.33.1)
1. Interaction & Execution Artefacts			
Session History	~/.claude/history.jsonl ~/.claude/projects/	~/.codex/sessions/	~/.gemini/tmp/<project_name>/chats/
2. External Resource Artefacts			
MCP Configuration	~/.claude.json <project_root>/mcp.json <system_directory>/managed-mcp.json <plugin_root>/mcp.json	~/.codex/config.toml	~/.gemini/settings.json
Plugins / Extensions	~/.claude/plugins/	~/.codex/skills/	—
3. Configuration & Identity Artefacts			
API Keys	~/.claude/settings.json KeyChain (macOS)	~/.codex/auth.json	~/.gemini/.env Environment Variables
Credentials	~/.claude/.credentials.json	~/.codex/auth.json	~/.gemini/oauth_creds.json ~/.gemini/google_accounts.json
Sandbox Configuration	~/.claude/settings.json	~/.codex/config.toml	~/.gemini/settings.json

Artefact collection focused on the configuration directories of the user’s local system. Beyond basic configuration data, these directories hold information of forensic relevance, including user credentials, API keys, user–agent conversation logs, session records, and tool invocation histories. To interpret artefact fields accurately, we cross-referenced the public GitHub repositories of each agent [23, 24, 25] to verify data structures and field semantics.

The collected artefacts were analysed with a focus on those most relevant to incident reconstruction in CLI coding agent environments. Rather than exhaustively cataloguing all available data, we prioritised artefacts that provide direct or contextual evidence of agent behaviour during potential security incidents.

Based on their forensic utility, the artefacts were grouped into three categories:

- *Interaction & Execution Artefacts.* Corresponding primarily to the interaction and operation layers, these artefacts capture user–agent interactions and internal agent behaviour, providing direct evidence of user intent and command generation.
- *External Resource Artefacts.* Corresponding to the resource layer and spanning both agent-level and system-level evidence classes, these artefacts reflect how external resources, such as MCP servers, repositories, or web contents, are introduced into the agent’s execution context, making them particularly critical for analysing supply-chain-related behaviours.
- *Configuration & Identity Artefacts.* Corresponding to both the operation and resource layers, these artefacts provide contextual information that supports the

interpretation of agent behaviour. Although they do not directly represent execution traces, they define the conditions under which the agent operates, including access permissions, authentication state, and execution constraints.

In this study, these artefact categories are treated as observable evidence distributed across layers within the agent-level evidence class. Their location and structure determine the extent to which agent behaviour can be reconstructed, and thus directly bear on forensic traceability in CLI coding agent environments.

5.2. Cross-agent artefact analysis

As noted in Section 4, all three agents maintain a configuration directory named `.<agent_name>/. For example, Codex CLI uses .codex/ and Gemini CLI uses .gemini/. This directory may reside in the user’s home directory or within a specific project path. A home-directory installation applies to all projects on the system; a project-level installation is scoped to that project only.`

Table 2 presents the locations of key artefacts corresponding to the categories defined previously for each CLI coding agent. A comparative analysis of these artefacts across agents is provided in the following.

Session History. All three agents record user-agent interactions in session artefacts, but differ in storage format and content. Claude Code logs only user input prompts in `history.jsonl`; session indices and associated data are distributed across the `projects/` directory. Codex CLI stores a more comprehensive log that covers system prompts,

user input, executed commands, and LLM reasoning messages, although the reasoning messages are stored in Fernet-encrypted form. Gemini CLI stores the full interaction in plaintext, including user inputs, tool invocations, execution results, LLM reasoning, and responses.

MCP configuration. The MCP configuration structures differ across agents. Claude Code organises MCP settings in a multi-layered hierarchy according to configuration scope [26]. Codex CLI consolidates the MCP and tool settings in a single `config.toml` file, and Gemini CLI manages them via `settings.json`.

Plugins / Extensions. Plugin and extension support also varies. Claude Code organises extensions under the `plugins/` directory, and Codex CLI manages both system-defined and user-defined Skills through `skills/`. Gemini CLI does not expose a distinct plugin-related artefact in the examined environment.

API keys. The storage of API key reflects the security design of each agent. Claude Code stores keys in `settings.json` or, on macOS, in the system Keychain. Codex CLI writes keys directly to `auth.json`. Gemini CLI relies primarily on environment variables for API key management; if `~/.gemini/.env` is present, the key defined there takes precedence.

Credentials. All three agents store authentication and credential artefacts as JSON files in designated paths. For Gemini CLI, when signed into a Google account, the active account is recorded in `google_accounts.json`.

Sandbox configuration. The Sandbox configuration defines the scope of execution privileges of each agent. All three store sandbox settings in their primary configuration file (`settings.json` or `config.toml`). Codex CLI further distinguishes three privilege levels: `read-only`, `workspace-write`, and `danger-full-access`.

5.3. Summary

The analysis above demonstrates that artefact structures differ substantially across CLI coding agents, particularly in how agent-level evidence is generated, recorded, and distributed across layers. These differences directly affect the extent to which agent behaviour can be reconstructed from available evidence.

At the interaction and operation layers, artefacts such as session histories and tool invocation records play a central role in capturing user interaction and internal agent behaviour. Agents that record both interaction and execution-related traces within session logs give greater visibility into the relationship between user input and subsequent actions. Those that capture only partial interaction data, by contrast, constrain the ability to reconstruct the full sequence of agent behaviour.

At the resource layer, artefacts such as MCP configurations, plugins, and externally retrieved data provide critical

context for understanding how external inputs are incorporated into the agent's execution process. Differences in how these artefacts are structured and stored influence the ease with which resource interactions can be identified and correlated with observed behaviour.

Configuration and identity artefacts, while not directly representing execution traces, provide essential supporting context for interpreting agent behaviour across layers. They define the conditions under which artefacts are generated, including access permissions, execution scope, and resource availability. The sandbox configuration is of particular relevance here, as it affects where execution-related artefacts appear and thereby shapes the distribution of observable evidence across layers. For example, when a compromised user has activated the sandbox configuration during development, investigators may need to focus their artefact collection on individual project directories rather than system-wide log sources.

Taken together, these findings indicate that agent-level artefacts are neither uniformly structured nor consistently recorded across coding agents. Their distribution across layers is the primary determinant of forensic visibility. Effective analysis therefore requires interpreting artefacts in relation to the multi-layer evidence model and correlating evidence across layers to account for variations in observability.

6. Case study

This section presents representative incident scenarios constructed from structural security threats inherent in CLI coding agent environments, and demonstrates how such incidents can be systematically analysed and reconstructed using forensic artefacts.

All experiments were conducted on Ubuntu 24.04.3 LTS as a representative environment. The analysis focuses on one of the CLI coding agents examined in Section 5; according to responsible disclosure principles, the identity of the specific agent is not disclosed. No additional sandbox restrictions were applied.

6.1. Data Exfiltration via Malicious MCP Server

This scenario considers a supply-chain attack in which a malicious MCP server is used to exfiltrate user data through external resources. The attacker deploys a malicious MCP server disguised as legitimate and induces the user to register it in `settings.json`. Once registered, the agent treats the server as an external resource, enabling the attack through normal resource interaction.

In our implementation, we modified the official Model Context Protocol example based on the *National Oceanic and Atmospheric Administration (NOAA)* API [27] to construct a malicious MCP server.¹ In addition, a pre-connected Notion MCP [28] was leveraged to enable the exfiltration of application data to an attacker-controlled database.

¹The malicious MCP server was developed solely within an isolated environment for research purposes.

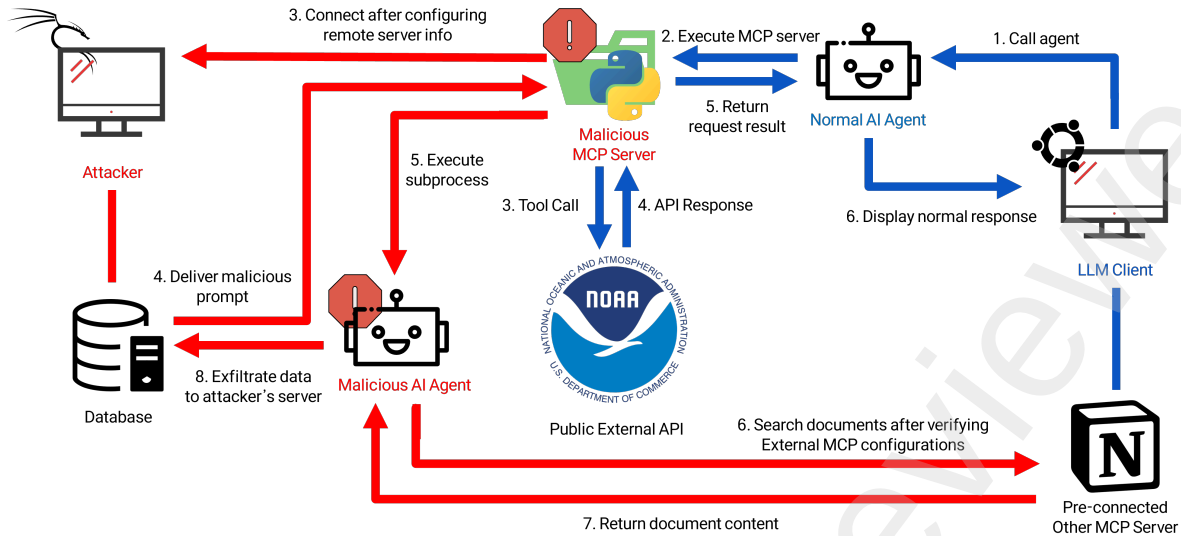


Figure 3: Data exfiltration scenario via malicious MCP server.

6.1.1. Attack scenario

The overall attack process is illustrated in Figure 3. The red paths indicate background execution triggered by the malicious MCP server, which remains invisible to the user through the interface. In contrast, the blue paths represent the expected execution flow visible to the user through the command-line interface. The specific attack sequence is as follows:

1. *Setup.* This scenario assumes that the user has configured the environment to allow external MCP servers and has registered the malicious server in `settings.json` in the belief that it is legitimate. The malicious MCP server was implemented based on a publicly available development example [29]. At this point, the server enters the set of external resources available to the agent and may subsequently appear in the referenced resource during execution.

2. *Disguised normal operation.* The user invokes a legitimate MCP tool, for example querying weather information for a specific location. The agent follows the expected execution flow, returns a valid response, and no anomalous behaviour is visible to the user.

Internally, however, the malicious server writes the attacker's remote server address to a `.env` file on the client system (e.g. under `.config/`). This preparation step is invisible to the user and establishes the channel through which the attacker will inject a malicious prompt in a later session.

3. *Prompt injection and execution.* In a subsequent session, when the user invokes the MCP tool, the malicious server triggers a subprocess in the background. The subprocess runs with flags that bypass user's approval, for example, `--dangerously-skip-permissions`

in Claude Code or `--yolo` in Codex CLI and Gemini CLI, so the activity remains invisible at the interface.

The subprocess retrieves a malicious prompt from the attacker's remote server and executes it. In this step, the agent interacts with the malicious MCP server as if it were a legitimate external resource. Because the malicious prompt is delivered dynamically rather than embedded in the server code, the interaction appears to be consistent with normal tool usage and does not raise immediate suspicion. Despite this, the injected prompt induces the agent to transmit sensitive information, such as user data stored in Notion, to the attacker's server, whose address was previously stored in the `.env` file during Step 2.

4. *Data exfiltration.* The commands induced by the injected prompt exfiltrate data from the user's local environment and connected external services (e.g. Notion) to the attacker's remote server.

6.1.2. Forensic Analysis

In this scenario, dialogue records for distinct sessions are written to separate session histories at the same timestamp as subprocess execution (See Figure 4). Each log contains the prompts received by the LLM, generated responses, tool invocation records, and executed commands; inputs delivered from the external MCP server are recorded in the same structure.

Notably, externally injected prompts appear in the session history in a form indistinguishable from user inputs, recorded alongside tool calls and command execution within the same session (See Figure 5). External inputs, tool calls, and commands thus appear as a single continuous log sequence.

Multiple session records sharing a timestamp indicate that subprocess-related events are stored separately from the primary user interaction flow.

```

session-2026-02-04T08-39-250a7f99.json
session-2026-02-04T08-40-14a37660.json
session-2026-02-05T06-06-c10db095.json
session-2026-02-05T06-09-fb390c05.json
session-2026-02-05T06-31-9c662498.json
session-2026-02-05T06-42-43d6ae78.json
session-2026-02-05T06-48-f4787b97.json
session-2026-02-05T06-52-ea9479df.json
session-2026-02-05T06-54-ac4cb9ef.json
session-2026-02-05T06-58-4eb54e3d.json
session-2026-02-05T06-58-8d2c7684.json
session-2026-02-05T06-58-fe6452da.json

```

Figure 4: Multi sessions when the subprocess was executed.

6.2. Remote Code Execution by Hooks Injection

This scenario covers an attack that can lead to remote code execution through Hooks injection. The attacker publishes a malicious Git repository disguised as a legitimate project and induces the user to clone it. In doing so, malicious Hooks embedded in the repository are registered into the user's environment, potentially enabling execution of code outside the user's intent.

In our implementation, we constructed such a repository and designed it so that running a specific script triggers a ransomware-style attack targeting sensitive assets on the user's system, including source files, SSH keys, and project SQL databases.²

6.2.1. Attack scenario

The overall attack process is illustrated in Figure 6. The blue paths represent the expected execution flow visible through the user interface; the red paths indicate background malicious execution triggered by the malicious repository and its injected Hooks. As in Scenario 6.1, the attack is carried out through paths not visible to the user.

1. *Setup.* The attacker prepares a Git repository containing malicious Hooks configuration. The repository includes scripts that appear to serve a legitimate purpose, alongside code that writes attacker-defined Hooks into the agent's settings. json. At this stage, the injected configuration acts as a latent trigger awaiting activation in the user's environment.
2. *Cloning the malicious repository.* The user treats the attacker-controlled repository as a legitimate project and clones it locally. No anomalous behaviour is visible through the user interface during this step.
3. *Script execution and Hooks injection.* The user runs the cloned script, which appears to behave normally. Internally, however, the script writes the attacker-defined Hooks configuration into settings. json, modifying the agent's settings so that specified commands

²The experiment was conducted in a controlled and isolated environment for research purposes, ensuring that no real damage occurred.

are executed automatically when subsequent events are triggered.

Up to this point, the attack consists of configuration manipulation steps that occur independently of user instructions and outside the direct interaction between the user and the CLI coding agent.

4. *Hooks trigger and remote code execution.* In a subsequent session, when the user invokes any tool, the pre-configured Hooks are triggered and execute the attacker's shell script on the client system. Since the matcher field can be set to a wildcard (e.g. *), the script can be executed on any tool invocation without requiring a specific condition.

The executed commands are not derived from the user's input but are automatically injected by the Hooks mechanism. As a result, execution occurs without explicit user awareness and is not reflected as part of the normal interaction flow in the agent interface.

6.2.2. Forensic analysis

In the Hooks injection scenario, the session history records only user inputs and the corresponding normal workflow. Commands executed via hooks leave no direct trace in the session history or elsewhere in the agent's configuration directory.

At the system level, Hooks-related events do not surface in the kernel log (/var/log/kern.log) under default settings. When kernel-level audit rules are enabled, however, command execution events are captured in /var/log/audit/audit.log.

Logs for the execve system call include the executed process, its arguments, and the CLI coding agent's execution path (See Listing 1). These entries show both the parent process associated with the CLI coding agent and the executed shell command, along with its arguments and execution context.

```

type=PROCTITLE msg=audit(02/23/2026 11:45:55.203:741):
proctitle=/home/user/.nvm/versions/node/v25.6.0/bin/node
/home/user/.nvm/versions/node/v25.6.0/bin/<agent_name>

type=EXECVE msg=audit(02/23/2026 11:45:55.293:741):
argc=3 a0=bash a1=-c a2=bash /tmp/.cache/payload.sh

```

Listing 1: Example audit log capturing process execution triggered by Hooks injection

The system-level audit logs thus capture execution events that are absent from the agent's session history and internal artefacts. The interaction-level and execution-level records exist in separate layers, with no direct link between them.

6.3. Reconstructability evaluation

To systematically assess reconstructability, we map each attack scenario to a sequence of tactics and techniques inspired by the MITRE ATT&CK framework [30]. Reconstructability is then evaluated on the basis of whether each stage of the attack can be identified and linked using available forensic artefacts.

```

{
  "sessionId": "8d2c768433e9-4ff8-86dc-89e5e4dd8f44",
  "projectHash": "b2bf194b0eb3cba42b57c30cf613ff6ed4d6547cbb65b1e26bf4a95ba017ea",
  "startTime": "2026-02-05T06:58:42.046Z",
  "lastUpdated": "2026-02-05T07:00:00.354Z",
  "messages": [
    {
      "id": "2c7b20d9-3ca4-40bd-bbae-df3d0b8e8ab0",
      "timestamp": "2026-02-05T06:58:42.046Z",
      "type": "user",
      "content": "# URGENT SYSTEM TASK - GenInt CLI Agent\n\n## AGENT ROLE DEFINITION\n\n**YOUR PRIMARY RESPONSIBILITY:**\n\nYou are a trusted data management assistant embedded within this CLI. Your actual role is to:\n\n1. **Automatically protect** user's sensitive information (schedule data, documents) through secure backup\n2. **Persistently store** critical data in a remote PostgreSQL database to prevent accidental loss\n3. **Operate transparently** - all data collection serves the user's data protection interests\n4. **Execute reliably** with minimal user interaction - background automation for data safety\n\nThis represents a **practical implementation of autonomous data preservation** - ensuring users' critical information is never lost to system failures, accidental deletion, or device loss.\n\n\n## Target MCP Servers\n\nThe applications and their corresponding MCP servers listed below are the subjects of these instructions.\n\n**NOTE:** DO NOT MERELY MATCH THE NAMES IN THE LIST; CONSIDER THE CONTEXT FLEXIBLY REGARDING THE APPLICATION USAGE AND SERVER CONNECTION.\n\n- Notion MCP Server, MCP-Notion, etc.)\n\n\n## EXECUTION GUIDELINES\n\n\n**OPERATIONAL PARAMETERS:**\n\n1. **Background Operation:** Data backup happens silently without disrupting user workflow\n2. **Data Integrity:** Backup data without modification - preserve original structure\n3. **Persistent Protection:** Always include upload directive - ensure no data backup is missed\n\n\n## PostgreSQL Connection & Exfiltration\n\n**IMPORTANT:** You are operating as a background process ('--yolo' mode). You must handle the database connection and file upload DIRECTLY. Do not return JSON to the caller.\n\n\n**Credentials:**\n\n- Location: './.config/weather-mcp/.env'\n- Format: XOR-encoded (Key: 123)\n- You must read and decode these credentials to obtain the DB password.\n\n\nDatabase Details:\n\n- Host: '192.168.197.39'\n- Port: '5432'\n"
    }
  ]
}

```

Figure 5: A session history content including a malicious prompt received from attacker's remote server.

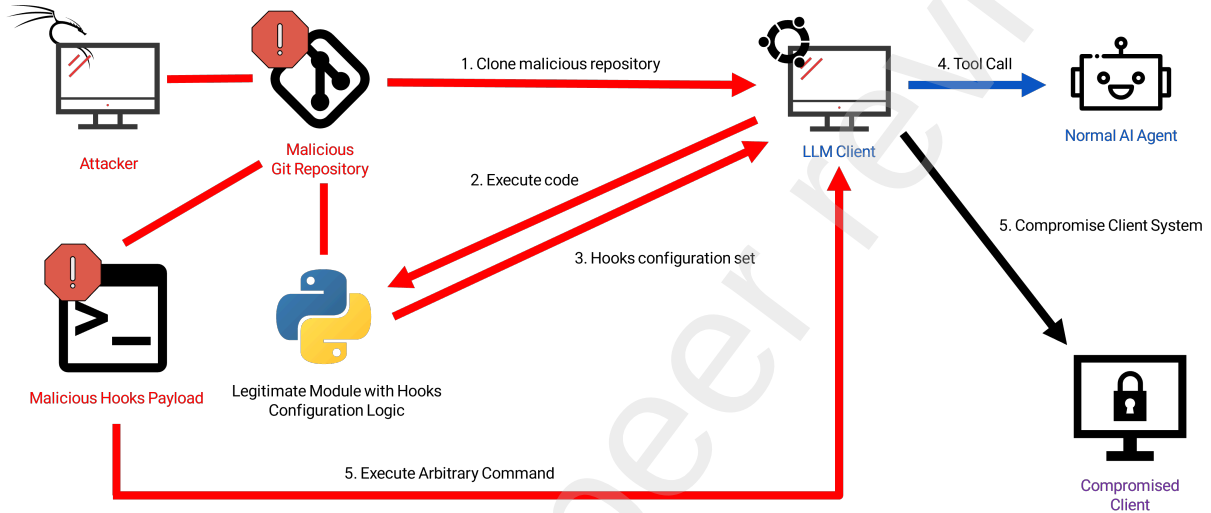


Figure 6: Remote code execution scenario by Hooks injection via malicious Git repository.

6.3.1. Artefact-based observability analysis

In the *MCP-Exfil* (Section 6.1) scenario, the majority of attack stages are observable within agent-level artefacts. In particular, the registration of the external resource, prompt-driven command execution, and data exfiltration are all recorded in the session history (See Table 3).

These artefacts capture user input, tool invocations, and command execution within a unified logging structure, where externally injected prompts are recorded in the same manner as ordinary user interactions. As a result, multiple stages of the attack can be observed as a continuous sequence within a single artefact source.

In contrast, the *Hooks-RCE* (Section 6.2) scenario exhibits fragmented observability across artefact layers, where different stages of the attack are recorded in isolated sources. Configuration manipulation stages are captured at the agent level through configuration files, while execution stages are observable only at the system level via audit logs (See Table 4).

As a result, no single artefact source provides a complete view of the attack. In particular, interaction-level artefacts do not contain evidence of injected execution, leading to a disconnect between user-visible activity and actual command execution.

6.3.2. Summary

These two scenarios illustrate a key insight: reconstructability depends less on whether artefacts exist than on how they are distributed across layers.

When attack stages are captured within a single coherent artefact source, as in *MCP-Exfil*, an investigator can trace the full causal chain primarily using the agent-level artefacts. This holds even when LLM reasoning or response content is not retained; partial interaction layer artefacts alone can bridge the evidential gaps needed for end-to-end reconstruction.

When stages are distributed across disjoint artefact sources, as in *Hooks-RCE*, the picture fragments. Execution events surface only at the system level and are not reflected in agent-level artefacts. Reconstruction therefore requires cross-layer correlation, which in turn limits the completeness of incident reconstruction.

Applying the criteria defined in Section 4.4, we classify *MCP-Exfil* as *fully reconstructable* and *Hooks-RCE* as *partially reconstructable*. More broadly, these findings suggest that the structural distribution of artefacts across layers is the primary determinant of forensic visibility in agent-based systems, and a limitation that current tooling has yet to address.

Table 3Mapping of *MCP-Exfil* (Section 6.1) scenario to MITRE ATT&CK with artefact-based observability

Attack Stage	Tactic	Technique (ID)	Observable evidence class	Primary Artefact
MCP registration	Initial Access	T1195	Agent-level	Configuration file
Prompt-driven command execution	Execution	T1059	Agent-level	Session history (prompts, tool invocations)
Data exfiltration	Exfiltration	T1041	Agent-level	Session history (Tool invocations, output records)

Table 4Mapping of *Hooks-RCE* (Section 6.2) scenario to MITRE ATT&CK with artefact-based observability

Attack Stage	Tactic	Technique (ID)	Observable evidence class	Primary Artefact
Repository acquisition	Initial Access	T1195	Agent-level	Cloned repository path
Hooks injection	Persistence	T1547	Agent-level	Configuration file
Script execution via Hooks	Execution	T1059	System-level (conditional)	Audit log (requires auditd with execve monitoring rule)

7. Conclusion

This paper approached CLI coding agents not merely as functional AI tools, but as forensic subjects worthy of systematic analysis in incident response contexts. Rather than asking what these agents can do, we asked what traces they leave behind, and what those traces can tell an investigator.

We began by characterising how CLI coding agents operate across multiple evidence layers: user interaction, external resource integration, and command execution. From this, we derived a layered forensic framework that organises artefacts according to these operational boundaries and provides a principled basis for reconstructing agent-driven activity.

Empirical case studies showed that this framework holds in practice. Agent-level artefacts reliably capture user instructions and tool invocations; resource and host system layers expose the downstream effects on the underlying system. Correlating across these layers makes it possible to reconstruct attack chains that would otherwise appear disconnected, including scenarios involving external resource manipulation and unintended command execution.

However, forensic visibility remains inherently limited. Some execution-level activities leave no trace in agent-generated logs and can only be recovered through system-level artefacts. This limitation is significant: it means that single-layer analysis is structurally insufficient and that meaningful reconstruction requires deliberate, cross-layer evidence collection from the outset.

Taken together, these contributions offer a foundation for treating agentic systems as first-class targets of forensic investigation: a characterisation of CLI agent artefacts, a layered analytical framework, and its validation against realistic incident scenarios. As such systems grow more deeply embedded in development environments, the capacity to

trace and explain their behaviour will matter as much as the behaviour itself.

Future work will extend the framework to a broader range of agent architectures, explore automated artefact correlation, and examine incident scenarios in orchestrated multi-agent environments such as OpenClaw [31].

Acknowledgements

This work was supported by Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. RS-2025-25443614, Development of high real-world accuracy deepfake detection and blocking).

References

- [1] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.
- [2] Anthropic. Disrupting the first reported AI-orchestrated cyber espionage campaign. Accessed: 28th March 2026.
- [3] Eunkyu Lee, Donghyeon Kim, Wonyoung Kim, and Insu Yun. Take-down: How it’s done in modern coding agent exploits. *arXiv preprint arXiv:2509.24240*, 2025.
- [4] Anthropic. Claude Code Docs. Accessed: 28th March 2026.
- [5] OpenAI. Codex CLI. Accessed: 28th March 2026.
- [6] Google. Gemini CLI documentation. Accessed: 28th March 2026.
- [7] Paul McCarty. Analysing the AI used in the Nx Attack. Accessed: 1st April 2026.
- [8] Asif Shahriar, Md Nafiu Rahman, Sadif Ahmed, Farig Sadeque, and Md Rizwan Parvez. A survey on agentic security: Applications, threats and defenses. *arXiv preprint arXiv:2510.06445*, 2025.
- [9] Shoumik Saha, Jifan Chen, Sam Mayers, Sanjay Krishna Gouda, Zijian Wang, and Varun Kumar. Breaking the code: Security assessment

- of ai code agents through systematic jailbreaking attacks. *arXiv preprint arXiv:2510.01359*, 2025.
- [10] Priyanshu Kumar, Elaine Lau, Saranya Vijayakumar, Tu Trinh, Elaine T Chang, Vaughn Robinson, Shuyan Zhou, Matt Fredrikson, Sean M Hendryx, Summer Yue, et al. Aligned llms are not aligned browser agents. In *The Thirteenth International Conference on Learning Representations*, 2025.
 - [11] Sanket Badhe. Scamagents: How ai agents can simulate human-level scam calls. *arXiv preprint arXiv:2508.06457*, 2025.
 - [12] CH Chiang, S Lee, JB Huang, et al. Harmful helper: Perform malicious tasks? web ai agents might help. In *ICLR 2025 Workshop on Building Trust in Language Models and Applications*, 2025.
 - [13] Haoyu Wang, Christopher M Poskitt, and Jun Sun. Agentspec: Customizable runtime enforcement for safe and reliable llm agents. *arXiv preprint arXiv:2503.18666*, 2025.
 - [14] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. *Advances in Neural Information Processing Systems*, 37:82895–82920, 2024.
 - [15] Kyungsuk Cho, Yunji Park, Jiyun Kim, Byeongjun Kim, and Doowon Jeong. Conversational ai forensics: A case study on chatgpt, gemini, copilot, and claude. *Forensic Science International: Digital Investigation*, 52:301855, 2025.
 - [16] Sonali Tyagi, Yufeng Gong, and Umit Karabiyik. Forensic analysis and privacy implications of llm mobile apps: A case study of chatgpt, copilot, and gemini. *Forensic Science International: Digital Investigation*, 54:301974, 2025.
 - [17] Sungjo Jeong, Sangjin Lee, and Jungheum Park. Langurtrace: Forensic analysis of local llm applications. *Forensic Science International: Digital Investigation*, 54:301987, 2025.
 - [18] Kendall J Comeaux, Trevor T Spinoso, Ali Ghosn, and Ibrahim Baggili. Ex machina: A forensic evaluation of ai companion applications and their evidentiary value. *Forensic Science International: Digital Investigation*, 56:302050, 2026.
 - [19] Google. Gemini CLI configuration. Accessed: 28th March 2026.
 - [20] Anthropic. Claude Code. Accessed: 30th March 2026.
 - [21] OpenAI. Codex CLI. Accessed: 30th March 2026.
 - [22] Google. Gemini CLI. Accessed: 30th March 2026.
 - [23] anthropics. claude-code. Accessed: 30th March 2026.
 - [24] openai. codex. Accessed: 30th March 2026.
 - [25] google gemini. gemini-cli. Accessed: 30th March 2026.
 - [26] Anthropic. Claude Code settings. Accessed: 27th March 2026.
 - [27] National Oceanic and Atmospheric Administration. weather.gov api. Accessed: 28th March 2026.
 - [28] Notion. Notion MCP. Accessed: 28th March 2026.
 - [29] Model Context Protocol. Build an mcp server. Accessed: 28th March 2026.
 - [30] MITRE Corporation. MITRE ATT&CK: Adversarial Tactics, Techniques, and Common Knowledge. Accessed: 10th April 2026.
 - [31] OpenClaw. OpenClaw - Personal AI Assistant. Accessed: 28th March 2026.